PROYECTO RENIEC - BID
Mejoramiento del acceso a los servicios de registros civiles e identificación de calidad a nivel nacional

ESTÁNDAR DE CODIFICACIÓN SEGURA

Mejoramiento de Acceso a los Servicios de Registros Civiles e Identificación de Calidad a Nivel Nacional (Contrato de Préstamo BID № 4297/OC-PE)

Componente 3: Incorporación de tecnologías para la prestación de los servicios

RENIEC
REGISTRO NACIONAL DE IDENTIFICACIÓN Y ESTADO CIVIL

Elaborado por:

Ing. Jarol Adolfo Bastidas Orihuela
Especialista en Arquitectura de Soluciones Senior

Ing. Gustavo Alfonso Montenegro Marcial
Especialista en Arquitectura de Soluciones Junior

Historial de modificaciones del Estándar de Codificación Segura

Versión	Fecha	Modificaciones	Preparado/Revisado por
1.0	24/06/2024	Versión inicial	Jarol Adolfo Bastidas Orihuela Gustavo Alfonso Montenegro Marcial
1.1	12/08/2025	Versión actualizada	Jarol Adolfo Bastidas Orihuela

FIRMA DIGITAL

Firmado digitalmente por: BASTIDAS ORIHUELA JAROL ADOLFO FIR 40561133 hard
Motivo: Soy el autor del documento
Fecha: 15/09/2025 09:36:31-0500

MIGUEL ÁNGEL MARTÍNEZ LA ROSA
Jefe de la Unidad de Ingeniería de Software
REGISTRO NACIONAL DE IDENTIFICACION Y ESTADO CIVIL

Firmado digitalmente por: MARTINEZ LA ROSA Miguel Angel FAU 20295613620 soft
Motivo: En señal de conformidad
Fecha: 15/09/2025 14:23:42-0500

1. OBJETIVO

Establecer un conjunto de buenas prácticas de codificación segura para garantizar una adecuada implementación de controles de seguridad durante el desarrollo de aplicaciones y de esa manera asegurar la confidencialidad, integridad y disponibilidad de la información.

2. ÁREA RESPONSABLE

Oficina de Tecnologías de la Información

i. **Unidad de Ingeniería de Software:** Es el responsable de la definición del estándar de desarrollo seguro, de mantenerla actualizada y de verificar el cumplimiento del estándar.
ii. **Líder del Equipo de Desarrollo:** Es el encargado de solicitar el soporte para cumplir con los controles de desarrollo seguro que se definen en el presente documento.

3. ALCANCE

El presente estándar aplica para su uso en la implementación del Sistema Integrado de Identificación y Registro Civil (SIIRC) y constituye además una guía de buenas prácticas para el desarrollo y mantenimiento de los sistemas de información a cargo de la Unidad de Ingeniería de Software (UIS).

4. REQUISITOS DE SEGURIDAD

El estándar está basado en OWASP Top 10 y OWASP API Security Top 10, por lo que es necesario que los equipos de desarrollo estén familiarizados con dichos estándares antes de iniciar el desarrollo del software.

4.1 Validación de entradas

a) Zero Trust: todo dato que provenga desde el cliente no es confiable y debe validarse en el backend.

b) Especificar conjuntos de caracteres adecuados, como UTF-8, para todas las fuentes de entrada.

c) Codificar datos en un juego de caracteres común antes de validar.

d) Validar todos los datos proporcionados por el cliente antes de procesarlos
Los datos por validar deben incluir:

- Cabeceras de la petición
- Parámetros recuperados del cuerpo de la petición
- Parámetros recuperados de la URI
- Datos de redirecciones

La validación de cada dato debe comprender:

- Validar el tipo de dato esperado
- Validar el rango de datos
- Validar la longitud del dato

- Validar el dato en base a una lista blanca de caracteres permitidos

e) La validación de datos debe realizarse en un sistema confiable (backend).

f) Los controles del lado del cliente (frontend) brindan poco o ningún beneficio de seguridad. Un atacante puede usar herramientas como proxies web del lado del cliente o herramientas de captura de paquetes de red para analizar el tráfico de aplicaciones y enviar solicitudes personalizadas, sin pasar por la interfaz de usuario por completo.

g) La aplicación tanto en el frontend como backend debe tener una rutina centralizada para la validación/sanitización de datos.

h) Toda falla de validación debe resultar en el rechazo de la entrada, con código y mensaje genérico.

i) No se debe permitir la inclusión de caracteres potencialmente peligrosos en la entrada de datos, a fin de prevenir ataques como SQL Injection, Cross-Site Scripting (XSS), etc. Se debe considerar, como mínimo, lo siguiente:

- Comprobar si hay bytes nulos (%00)
- Evitar los siguientes caracteres: (< > " ' % () & + \ \' \")
- Comprobar si hay nuevos caracteres de línea (%0d, %0a, \r, \n)
- Comprobar si hay caracteres de alteraciones de rutas (../ o ..\)

4.2 Codificación de salidas

a) Realizar toda codificación en un sistema confiable (backend).

b) La aplicación debe tener una rutina centralizada para cada tipo de codificación.

c) Codificar todos los datos devueltos al cliente.

d) Sanitizar toda salida de datos que no son de confianza antes de realizar consultas al SO y/o consultas del tipo SQL, XML y LDAP.

4.3 Autenticación y administración de contraseñas

a) El consumo de cualquier recurso debe exigir una autenticación previa.

b) Dependiendo de la necesidad de autenticación de la aplicación, esta deberá utilizar un componente de seguridad estándar definido en el documento “Estándar de Autenticación y Autorización”.

c) No divulgar información confidencial en respuestas de error por fallas de autenticación, como, por ejemplo, el lenguaje de programación o la base de datos.

d) Las respuestas de error de autenticación no deben indicar qué parte de los datos de autenticación era incorrecta. Por ejemplo, en lugar de "Nombre de usuario no válido" o "Contraseña no válida", simplemente use "Nombre de usuario y/o contraseña no válidos" para ambos.

e) Toda conexión con sistemas externos debe exigir autenticación. El tipo de autenticación debe estar alineado al estándar de autenticación definido en el documento “Estándar de Autenticación y Autorización”.

f) El campo de entrada para la contraseña en los formularios web debe ser del tipo “password” y tener deshabilitada la funcionalidad “recordarme” (autocomplete=off).

g) Utilice solo solicitudes HTTPS POST para transmitir credenciales de autenticación.

h) Si la aplicación administra un almacén de credenciales, debe asegurarse de que solo se almacenen hashes criptográficamente seguros (ver documento “Estándar de Criptografía”) de las contraseñas.

i) El hash de contraseña debe implementarse en un sistema de confianza (backend).

4.4 Gestión de sesiones

a) La creación del identificador de un token de sesión siempre debe realizarse en un sistema de confianza, tales como un Proveedor de identidad (IdP) o un backend.

b) Debe asegurarse que la funcionalidad “Cerrar sesión” termine completamente la sesión en el IdP o backend donde se generó e incluso elimine las cookies generadas para ese fin.

c) Debe establecer un tiempo de vida del token de sesión que vaya acorde a las funcionalidades y datos expuestos en la aplicación. Por el momento, este tiempo deberá ser definido por el equipo de Arquitectura en base a un análisis de riesgos de la aplicación.

d) No exponga identificadores de sesión en direcciones URL, mensajes de error o logs. Los identificadores de sesión solo deben ubicarse en el encabezado de cookie HTTP.

e) Establecer los siguientes atributos a fin de asegurar las cookies:

- Httponly: permite que la cookie y su respectivo valor no puedan ser leídos mediante código script
- Secure: evita que la cookie sea transportada por un medio no seguro (texto plano)
- Domain: limita el uso de la cookie a un dominio específico
- Path: limita el uso de la cookie a una ruta específica
- Expires: se verifica si las cookies son persistentes y en ese caso que tipo de información almacenan

4.5 Control de acceso

a) Utilice datos extraídos de la sesión para validar la autorización.

b) Dependiendo de la necesidad de autorización de la aplicación, esta deberá utilizar un componente de seguridad estándar definido en el documento “Estándar de Autenticación y Autorización”.

c) Aplique controles de autorización en cada solicitud, incluidos los realizados por scripts (batch) del lado del servidor.

d) Restrinja el acceso a archivos, datos de la aplicación, atributos del usuario, información de políticas, configuraciones, servicios u otros recursos, incluidos los que están fuera del control directo de la aplicación, solo a los usuarios autorizados.

e) Limite el número de transacciones que un solo usuario o dispositivo puede realizar en un período de tiempo determinado a fin de prevenir los ataques automatizados.

f) Cuentas de servicio o cuentas que admitan conexiones hacia o desde sistemas externos deben tener el menor privilegio posible.

4.6 Prácticas criptográficas:

a) Proteja la información confidencial y secretos con las técnicas estándares de protección de datos definido en el documento “Estándar de controles criptográficos”.

b) Proteja los secretos maestros del acceso no autorizado.

c) Todo valor aleatorio, ya sea números, nombres de archivo, GUID o cadenas, debe ser generado mediante algoritmos no deterministas. De esta forma se asegura que los valores aleatorios sean criptográficamente seguros. Utilice la siguiente librería:

- Para Java: SecureRandom

4.7 Gestión de errores y logs

a) No divulgue información confidencial en respuestas de error, incluidos los detalles del sistema, identificadores de sesión o información de la cuenta.

b) Implemente mensajes de error genéricos y use páginas de error personalizadas.

c) La aplicación debe controlar los errores de la aplicación y no depender de la configuración del servidor.

d) Utilice una rutina maestra o librería para el registro de eventos en los logs.

e) No almacene información confidencial en los logs, incluidos los detalles innecesarios del sistema, los identificadores de sesión o las contraseñas. Para mayor detalle revisar el documento “Estándar de Protección de datos confidenciales”.

f) Asegúrese de que el registro de eventos en los logs contemple lo siguiente:

- Registre tanto casos de éxito como casos de error
- Registre todos los errores de validación de entrada
- Registre todos los intentos de autenticación
- Registre todos los errores de control de acceso
- Registre intentos fallidos de conexión ya sea por tokens de sesión no válidos o caducados
- Registre todas las excepciones del sistema
- Registre todas las funciones administrativas, incluidos los cambios en los valores de configuración de seguridad
- Registre todos los errores de conexión TLS del backend
- Registre errores del módulo criptográfico

g) Campos mínimos necesarios que debe registrar en los Logs:

- ID de la transacción
- ID de aplicación
- Nombre de la aplicación
- ID del usuario consumidor
- Fecha
- Hora

4.8 Protección de datos

a) La información recuperada de las APIs debe ser la mínima e indispensable para poder ejecutar la funcionalidad.

b) Implemente privilegios mínimos, restrinja a los usuarios solo la funcionalidad, los datos y la información del sistema necesarios para realizar sus tareas.

c) Proteja los datos confidenciales almacenados en caché o temporales de accesos no autorizados y eliminé estos archivos o registros tan pronto como ya no sean necesarios.

d) No almacene del lado del cliente o en el código fuente:

- Contraseñas en claro
- Cadenas de conexión
- Credenciales de autenticación
- Tokens de acceso (salvo haya sido generado después de un proceso de autenticación y esté relacionado a un token de sesión específico)
- Claves privadas de cifrado
- Información confidencial

e) No incluya información confidencial en los parámetros de solicitud HTTPS GET.

f) Implemente controles de acceso adecuados para los datos confidenciales almacenados en el backend. Esto incluye datos almacenados en caché, archivos temporales y datos a los que solo pueden acceder usuarios específicos del sistema y/o administradores.

g) A fin de proteger la información confidencial, ya sea en transmisión o en reposo, aplique las técnicas de protección de datos (dependiendo del caso de uso) descritas en el documento “Estándar de protección de datos confidenciales”.

4.9 Seguridad de la comunicación

a) Asegúrese de que toda conexión que utilice la aplicación haga uso de protocolo HTTPS.

b) Asegúrese de utilizar conexiones TLS 1.3 o superior para todo el contenido que requiere acceso autenticado y para toda otra información confidencial.

c) Asegúrese de utilizar TLS 1.3 o superior para conexiones a sistemas externos.

4.10 Configuración del sistema

a) Elimine o deshabilite todas las funcionalidades y/o archivos innecesarios.

b) Elimine código de prueba o cualquier funcionalidad no destinada a producción, antes de la implementación.

c) Elimine información innecesaria de los encabezados de respuesta HTTP relacionados con el sistema operativo, la versión del servidor web, marcos de aplicación, lenguaje de programación, tipo y versión de base de datos, gestor de contenidos, entre otros.

4.11 Seguridad de la base de datos

a) Utilice las prácticas de validación de entrada y codificación de salida y asegúrese de abordar los caracteres especiales. Si estos fallan, no ejecute la sentencia en la base de datos.

b) Los microservicios deben utilizar el nivel más bajo posible de privilegios al acceder a la base de datos.

c) Las cadenas de conexión no deben codificarse de forma rígida dentro de la aplicación. Las cadenas de conexión deben almacenarse en un archivo de configuración independiente en un sistema de confianza y deben cifrarse.

4.12 Administración de archivos

a) Solicite autenticación antes de permitir que se cargue un archivo en su aplicación.

b) Limite el tipo de archivos que se pueden cargar solo a los tipos que son necesarios para satisfacer el flujo de negocio. Utilice una lista blanca para tal fin.

c) Valide que los archivos cargados son del tipo que espera su aplicación comprobando los encabezados de archivo. La comprobación del tipo de archivo por extensión por sí solo no es suficiente.

d) Desactive los privilegios de ejecución en directorios de carga de archivos.

e) Nunca envíe la ruta de acceso absoluta del archivo al frontend.

f) Analice los archivos subidos por el usuario en busca de virus y malware.